

Отчёт по лабораторной работе №5

Управление системными службами

Лабси Мохаммед

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Управление службой Very Secure FTP	6
2.2	Разрешение конфликтов между firewalld и iptables	11
2.3	Работа с изолируемыми целями systemd	16
2.4	Настройка цели systemd по умолчанию	17
2.5	Контрольные вопросы	19
3	Заключение	22

Список иллюстраций

2.1	Установка, запуск и проверка состояния службы vsftpd	7
2.2	Добавление службы vsftpd в автозапуск	7
2.3	Удаление службы vsftpd из автозапуска	8
2.4	Проверка символической ссылки автозапуска и состояния vsftpd	9
2.5	Просмотр зависимостей службы vsftpd	10
2.6	Просмотр обратных зависимостей службы vsftpd	10
2.7	Проверка состояния служб firewalld и iptables	11
2.8	Демонстрация конфликта между firewalld и iptables	12
2.9	Содержимое файла юнита firewalld.service	13
2.10	Содержимое файла юнита iptables.service	14
2.11	Маскирование службы iptables и проверка блокировки её запуска	15
2.12	Поиск целей systemd, поддерживающих изоляцию	16
2.13	Работа в rescue mode и переход к цели reboot.target	17
2.14	Изменение цели по умолчанию на multi-user.target	18
2.15	Проверка текстового режима и восстановление graphical.target как цели по умолчанию	19

Список таблиц

1 Цель работы

Получить навыки управления системными службами операционной системы посредством systemd.

2 Выполнение лабораторной работы

2.1 Управление службой Very Secure FTP

Получаю полномочия администратора и устанавливаю службу Very Secure FTP с помощью пакетного менеджера DNF:

```
dnf -y install vsftpd
```

После установки пакет `vsftpd-3.0.5-12.el10.x86_64` успешно добавлен в систему.

Для запуска службы первоначально была введена команда:

```
dnf -y start vsftpd
```

DNF сообщает, что команды `start` не существует, поскольку запуск и остановка системных служб выполняются не средствами пакетного менеджера, а с помощью `systemctl`. Поэтому запускаю службу корректной командой:

```
systemctl start vsftpd
```

После запуска проверяю её состояние:

```
systemctl status vsftpd
```

Служба `vsftpd.service` находится в состоянии `active (running)`, то есть FTP-сервер запущен и работает. При этом параметр `Loaded` показывает состояние `disabled`, следовательно, служба пока не добавлена в автоматический запуск при загрузке операционной системы (см. рис. Рисунок 2.1).

```

Installed:
  vsftpd-3.0.5-12.el10.x86_64

Complete!
root@mohammedlabsi:/home/mohammedlabsi#
root@mohammedlabsi:/home/mohammedlabsi# dnf -y start vsftpd
No such command: start. Please use /bin/dnf --help
It could be a DNF plugin command, try: "dnf install 'dnf-command(start)'"
root@mohammedlabsi:/home/mohammedlabsi# systemctl start vsftpd
root@mohammedlabsi:/home/mohammedlabsi# systemctl status vsftpd
● vsftpd.service - Vsftpd ftp daemon
   Loaded: loaded (/usr/lib/systemd/system/vsftpd.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-09-26 13:42:50 MSK; 12s ago
  Invocation: a0f76f47b4f146a49856e5a245a44144
    Process: 3882 ExecStart=/usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf (code=exited, status=0/SUCCESS)
   Main PID: 3883 (vsftpd)
      Tasks: 1 (limit: 24637)
     Memory: 868K (peak: 1.2M)
        CPU: 2ms
    CGroup: /system.slice/vsftpd.service
            └─3883 /usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf

Sep 26 13:42:50 mohammedlabsi systemd[1]: Starting vsftpd.service - Vsftpd ftp daemon...
Sep 26 13:42:50 mohammedlabsi systemd[1]: Started vsftpd.service - Vsftpd ftp daemon.
root@mohammedlabsi:/home/mohammedlabsi# c

```

Рисунок 2.1: Установка, запуск и проверка состояния службы vsftpd

Добавляю службу vsftpd в автозапуск:

```
systemctl enable vsftpd
```

В результате создаётся символическая ссылка:

```
/etc/systemd/system/multi-user.target.wants/vsftpd.service → /usr/lib/systemd/system/v
```

Повторно проверяю состояние:

```
systemctl status vsftpd
```

Теперь в строке Loaded отображается состояние enabled, а сама служба продолжает находиться в состоянии active (running). Это означает, что vsftpd работает в текущем сеансе и будет автоматически запускаться при последующих загрузках системы (см. рис. Рисунок 2.2).

```

root@mohammedlabsi:/home/mohammedlabsi# systemctl enable vsftpd
Created symlink '/etc/systemd/system/multi-user.target.wants/vsftpd.service' → '/usr/lib/systemd/system/vsftpd.service'.
root@mohammedlabsi:/home/mohammedlabsi# systemctl status vsftpd
● vsftpd.service - Vsftpd ftp daemon
   Loaded: loaded (/usr/lib/systemd/system/vsftpd.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-09-26 13:42:50 MSK; 55s ago
  Invocation: a0f76f47b4f146a49856e5a245a44144
    Main PID: 3883 (vsftpd)
      Tasks: 1 (limit: 24637)
     Memory: 868K (peak: 1.2M)
        CPU: 2ms
    CGroup: /system.slice/vsftpd.service
            └─3883 /usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf

Sep 26 13:42:50 mohammedlabsi systemd[1]: Starting vsftpd.service - Vsftpd ftp daemon...
Sep 26 13:42:50 mohammedlabsi systemd[1]: Started vsftpd.service - Vsftpd ftp daemon.
root@mohammedlabsi:/home/mohammedlabsi#
root@mohammedlabsi:/home/mohammedlabsi# █

```

Рисунок 2.2: Добавление службы vsftpd в автозапуск

Для удаления службы из автозапуска выполняю:

```
systemctl disable vsftpd.service
```

После выполнения команды созданная ранее символическая ссылка удаляется. Проверяю состояние службы:

```
systemctl status vsftpd
```

Параметр Loaded снова принимает значение disabled. При этом состояние остаётся active (running), поскольку команда disable не останавливает уже запущенную службу, а только исключает её автоматический запуск при следующей загрузке операционной системы (см. рис. Рисунок 2.3).

```
root@mohammedlabsi:/home/mohammedlabsi# systemctl disable vsftpd.service
Removed '/etc/systemd/system/multi-user.target.wants/vsftpd.service'.
root@mohammedlabsi:/home/mohammedlabsi# systemctl status vsftpd
● vsftpd.service - Vsftpd ftp daemon
   Loaded: loaded (/usr/lib/systemd/system/vsftpd.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-09-26 13:42:50 MSK; 1min 34s ago
 Invocation: a0f76f47b4f146a49856e5a245a44144
    Main PID: 3883 (vsftpd)
      Tasks: 1 (limit: 24637)
     Memory: 868K (peak: 1.2M)
        CPU: 2ms
    CGroup: /system.slice/vsftpd.service
            └─3883 /usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf

Sep 26 13:42:50 mohammedlabsi systemd[1]: Starting vsftpd.service - Vsftpd ftp daemon...
Sep 26 13:42:50 mohammedlabsi systemd[1]: Started vsftpd.service - Vsftpd ftp daemon.
root@mohammedlabsi:/home/mohammedlabsi#
```

Рисунок 2.3: Удаление службы vsftpd из автозапуска

Проверяю содержимое каталога символических ссылок для цели multi-user.target:

```
ls /etc/systemd/system/multi-user.target.wants/
```

После выполнения disable ссылка vsftpd.service в каталоге отсутствует.

Снова добавляю службу в автозапуск:

```
systemctl enable vsftpd
```

Повторно просматриваю содержимое каталога:

```
ls /etc/systemd/system/multi-user.target.wants/
```

В списке появляется vsftpd.service, что подтверждает создание ссылки, отвечающей за автоматический запуск службы при переходе системы к цели multi-user.target.

Для дополнительной проверки выполняю:

```
systemctl status vsftpd
```

Состояние загрузки юнита изменилось на `enabled`, а состояние самой службы остаётся `active (running)` (см. рис. Рисунок 2.4).

```
root@mohammedlabsl:/home/mohammedlabsl# ls /etc/systemd/system/multi-user.target.wants/
atd.service      crond.service      libstorageengine.service  remote-cryptsetup.target  sssd.service
auditd.service   cups.path           mcelog.service           remote-fs.target          tuned.service
audit-rules.service  cups.service       mdmmonitor.service       rsyslog.service          vboxadd.service
avahi-daemon.service  firewallld.service  ModemManager.service    smartd.service           vboxadd-service.service
chronyd.service     irqbalance.service  NetworkManager.service  sshd.service             vmttoolsd.service
root@mohammedlabsl:/home/mohammedlabsl# systemctl enable vsftpd
Created symlink '/etc/systemd/system/multi-user.target.wants/vsftpd.service' → '/usr/lib/systemd/system/vsftpd.service'.
root@mohammedlabsl:/home/mohammedlabsl# ls /etc/systemd/system/multi-user.target.wants/
atd.service      cups.path           mdmmonitor.service       smartd.service          vmttoolsd.service
auditd.service   cups.service       ModemManager.service    sshd.service           vsftpd.service
audit-rules.service  firewallld.service  NetworkManager.service  sssd.service           vboxadd.service
avahi-daemon.service  irqbalance.service  remote-cryptsetup.target  tuned.service          vboxadd-service.service
chronyd.service     libstorageengine.service  remote-fs.target          vboxadd.service
crond.service       mcelog.service      rsyslog.service
root@mohammedlabsl:/home/mohammedlabsl# systemctl status vsftpd
● vsftpd.service - Vsftpd ftp daemon
   Loaded: loaded (/usr/lib/systemd/system/vsftpd.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-09-26 13:42:50 MSK; 2min 48s ago
  Invocation: a0f76f47b4f146a49856e5a245a44144
    Main PID: 3883 (vsftpd)
      Tasks: 1 (limit: 24637)
     Memory: 868K (peak: 1.2M)
        CPU: 2ms
    CGroup: /system.slice/vsftpd.service
            └─3883 /usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf

Sep 26 13:42:50 mohammedlabsl systemd[1]: Starting vsftpd.service - Vsftpd ftp daemon...
Sep 26 13:42:50 mohammedlabsl systemd[1]: Started vsftpd.service - Vsftpd ftp daemon.
root@mohammedlabsl:/home/mohammedlabsl#
```

Рисунок 2.4: Проверка символической ссылки автозапуска и состояния vsftpd

Для просмотра зависимостей службы выполняю:

```
systemctl list-dependencies vsftpd
```

Команда формирует иерархический список юнитов, от которых зависит `vsftpd.service`. Среди них присутствуют `system.slice`, `sysinit.target`, различные файловые системы, сокеты, системные службы и другие компоненты, необходимые для работы сервиса. Зелёными маркерами обозначены активные зависимости, а пустыми — неактивные на момент выполнения команды (см. рис. Рисунок 2.5).

```

root@mohammedlabsi:/home/mohammedlabsi# systemctl list-dependencies vsftpd
vsftpd.service
● └─system.slice
● └─sysinit.target
● └─dev-hugepages.mount
● └─dev-mqueue.mount
● └─dracut-shutdown.service
○ └─fips-crypto-policy-overlay.service
○ └─iscsi-onboot.service
○ └─iscsi-starter.service
● └─kmod-static-nodes.service
○ └─ldconfig.service
● └─lvm2-lvmpolld.socket
● └─lvm2-monitor.service
○ └─multipathd.service
● └─plymouth-read-write.service
● └─plymouth-start.service
● └─proc-sys-fs-binfmt_misc.automount
○ └─selinux-autorelabel-mark.service
● └─sys-fs-fuse-connections.mount
● └─sys-kernel-config.mount

```

Рисунок 2.5: Просмотр зависимостей службы vsftpd

Для просмотра обратных зависимостей, то есть юнитов, зависящих от vsftpd, использую:

```
systemctl list-dependencies vsftpd --reverse
```

В результате видно, что vsftpd.service связан с multi-user.target, которая, в свою очередь, входит в зависимости graphical.target. Следовательно, при обычной многопользовательской или графической загрузке системы включённая служба может быть запущена как часть соответствующей цели (см. рис. Рисунок 2.6).

```

○ └─system.slice
root@mohammedlabsi:/home/mohammedlabsi# systemctl list-dependencies vsftpd --reverse
vsftpd.service
● └─multi-user.target
● └─graphical.target
root@mohammedlabsi:/home/mohammedlabsi# █

```

Рисунок 2.6: Просмотр обратных зависимостей службы vsftpd

2.2 Разрешение конфликтов между firewalld и iptables

Для выполнения задания устанавливаю пакеты iptables, включая необходимые дополнительные компоненты:

```
dnf -y install iptables\*
```

После завершения установки проверяю состояние firewalld:

```
systemctl status firewalld.service
```

Служба firewalld.service находится в состоянии active (running) и имеет состояние загрузки enabled.

Затем проверяю iptables:

```
systemctl status iptables.service
```

Служба iptables.service имеет состояние загрузки disabled и в данный момент находится в состоянии inactive (dead) (см. рис. Рисунок 2.7).

```
Installed:
  iptables-devel-1.8.11-16.el10_2.x86_64 iptables-nft-services-1.8.11-16.el10_2.noarch iptables-utils-1.8.11-16.el10_2

Complete!
root@mohammedlabsi:/home/mohammedlabsi#
root@mohammedlabsi:/home/mohammedlabsi#
root@mohammedlabsi:/home/mohammedlabsi# systemctl status firewalld.service
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-09-26 13:40:18 MSK; 7min ago
  Invocation: b92a0c404dd6485b9be64470d3921204
     Docs: man:firewalld(1)
    Main PID: 1087 (firewalld)
       Tasks: 2 (limit: 24637)
      Memory: 47.6M (peak: 49.5M)
         CPU: 266ms
    CGroup: /system.slice/firewalld.service
            └─1087 /usr/bin/python3 -sP /usr/sbin/firewalld --nofork --nopid

Sep 26 13:40:18 mohammedlabsi systemd[1]: Started firewalld.service - firewalld - dynamic firewall daemon.
root@mohammedlabsi:/home/mohammedlabsi# systemctl status iptables.service
○ iptables.service - IPv4 firewall with iptables
   Loaded: loaded (/usr/lib/systemd/system/iptables.service; disabled; preset: disabled)
   Active: inactive (dead)
root@mohammedlabsi:/home/mohammedlabsi#
```

Рисунок 2.7: Проверка состояния служб firewalld и iptables

Для демонстрации конфликта пытаюсь последовательно запустить обе службы:

```
systemctl start firewalld
```

```
systemctl start iptables.service
```

После запуска iptables снова проверяю состояние firewalld:

```
systemctl status firewalld.service
```

Служба firewalld была автоматически остановлена и перешла в состояние inactive (dead).

Проверяю состояние iptables:

```
systemctl status iptables.service
```

Для iptables.service отображается состояние active (exited). Такое состояние связано с типом юнита oneshot: команда настройки правил выполняется и завершается, после чего systemd считает юнит активным благодаря параметру RemainAfterExit=yes.

Таким образом, одновременно использовать firewalld и классическую службу iptables нельзя: при запуске конфликтующего юнита ранее активная служба firewalld деактивируется (см. рис. Рисунок 2.8).

```
-----
root@mohammedlabsi:/home/mohammedlabsi# systemctl start firewalld
root@mohammedlabsi:/home/mohammedlabsi# systemctl start iptables.service
root@mohammedlabsi:/home/mohammedlabsi# systemctl status firewalld.service
○ firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; preset: enabled)
   Active: inactive (dead) since Sat 2026-09-26 13:48:55 MSK; 2s ago
     Duration: 8min 37.337s
  Invocation: b92a0c404dd6485b9be64470d3921204
       Docs: man:firewalld(1)
   Process: 1087 ExecStart=/usr/sbin/firewalld --nofork --nopid $FIREWALLD_ARGS (code=exited, status=0/SUCCESS)
    Main PID: 1087 (code=exited, status=0/SUCCESS)
   Memory peak: 49.5M
      CPU time: 281ms

Sep 26 13:40:18 mohammedlabsi systemd[1]: Started firewalld.service - firewalld - dynamic firewall daemon.
Sep 26 13:48:55 mohammedlabsi systemd[1]: Stopping firewalld.service - firewalld - dynamic firewall daemon...
Sep 26 13:48:55 mohammedlabsi systemd[1]: firewalld.service: Deactivated successfully.
Sep 26 13:48:55 mohammedlabsi systemd[1]: Stopped firewalld.service - firewalld - dynamic firewall daemon.
root@mohammedlabsi:/home/mohammedlabsi# systemctl status iptables.service
● iptables.service - IPv4 firewall with iptables
   Loaded: loaded (/usr/lib/systemd/system/iptables.service; disabled; preset: disabled)
   Active: active (exited) since Sat 2026-09-26 13:48:55 MSK; 7s ago
  Invocation: 60f5cdcdc824ff0aaebbd234886a516
   Process: 5464 ExecStart=/usr/libexec/iptables/iptables.init start (code=exited, status=0/SUCCESS)
    Main PID: 5464 (code=exited, status=0/SUCCESS)
   Memory peak: 1.5M
      CPU time: 6ms

Sep 26 13:48:55 mohammedlabsi systemd[1]: Starting iptables.service - IPv4 firewall with iptables...
Sep 26 13:48:55 mohammedlabsi iptables.init[5464]: iptables: Applying firewall rules: [ OK ]
Sep 26 13:48:55 mohammedlabsi systemd[1]: Finished iptables.service - IPv4 firewall with iptables.
root@mohammedlabsi:/home/mohammedlabsi#
```

Рисунок 2.8: Демонстрация конфликта между firewalld и iptables

Для определения причины конфликта просматриваю файл юнита firewalld:

```
cat /usr/lib/systemd/system/firewalld.service
```

В секции [Unit] присутствует строка:

```
Conflicts=iptables.service ip6tables.service ebtables.service ipset.service
```

Директива Conflicts указывает systemd, что firewalld.service не должен работать одновременно с перечисленными юнитами. В данном случае к ним относятся iptables.service, ip6tables.service, ebtables.service и ipset.service.

Также в конфигурации присутствуют параметры Before=network-pre.target и Wants=network-pre.target, определяющие связь службы с ранним этапом настройки сети. В секции [Install] указано WantedBy=multi-user.target, что позволяет подключать службу к стандартной многопользовательской цели системы (см. рис. Рисунок 2.9).

```
root@mohammedlabsi:/home/mohammedlabsi# cat /usr/lib/systemd/system/firewalld.service
[Unit]
Description=firewalld - dynamic firewall daemon
Before=network-pre.target
Wants=network-pre.target
# Use PartOf so firewalld is restarted when dbus is restarted. But it doesn't
# have to wait on it initially.
PartOf=dbus.service
Conflicts=iptables.service ip6tables.service ebtables.service ipset.service
Documentation=man:firewalld(1)

[Service]
EnvironmentFile=-/etc/sysconfig/firewalld
ExecStart=/usr/sbin/firewalld --nofork --nopid $FIREWALLD_ARGS
ExecReload=/bin/kill -HUP $MAINPID
StandardOutput=null
StandardError=null
KillMode=mixed
DevicePolicy=closed
KeyringMode=private
LockPersonality=yes
MemoryDenyWriteExecute=yes
PrivateDevices=yes
ProtectClock=yes
ProtectControlGroups=yes
ProtectHome=tmpfs
ProtectHostname=yes
ProtectKernelLogs=yes
ProtectKernelModules=no
ProtectKernelTunables=no
ProtectSystem=yes
RestrictRealtime=yes
RestrictSUIDSGID=yes
SystemCallArchitectures=native

[Install]
WantedBy=multi-user.target
Alias=dbus-org.fedoraproject.FirewallD1.service
root@mohammedlabsi:/home/mohammedlabsi#
```

Рисунок 2.9: Содержимое файла юнита firewalld.service

Просматриваю файл юнита iptables:

```
cat /usr/lib/systemd/system/iptables.service
```

В секции [Unit] присутствуют параметры:

```
Wants=network-pre.target
```

Before=network-pre.target

Явной директивы Conflicts=firewalld.service в данном файле нет. Следовательно, взаимное исключение обеспечивается директивой Conflicts со стороны юнита firewalld.service.

В секции [Service] установлен параметр:

Type=oneshot

Команда запуска задаётся директивой:

ExecStart=/usr/libexec/iptables/iptables.init start

Также присутствует:

RemainAfterExit=yes

Поэтому после выполнения скрипта настройки правил процесс завершается, однако юнит продолжает считаться активным. Команды ExecReload и ExecStop используются соответственно для перезагрузки и остановки правил iptables (см. рис. Рисунок 2.10).

```
root@monammedlabs1:/home/monammedlabs1#  
root@monammedlabs1:/home/mohammedlabs1# cat /usr/lib/systemd/system/iptables.service  
[Unit]  
Description=IPv4 firewall with iptables  
AssertPathExists=/etc/sysconfig/iptables  
Before=network-pre.target  
Wants=network-pre.target  
  
[Service]  
Type=oneshot  
RemainAfterExit=yes  
ExecStart=/usr/libexec/iptables/iptables.init start  
ExecReload=/usr/libexec/iptables/iptables.init reload  
ExecStop=/usr/libexec/iptables/iptables.init stop  
Environment=BOOTUP=serial  
Environment=CONSOLETYPE=serial  
  
[Install]  
WantedBy=multi-user.target  
root@monammedlabs1:/home/mohammedlabs1#
```

Рисунок 2.10: Содержимое файла юнита iptables.service

Для предотвращения случайного запуска iptables сначала останавливаю эту службу и запускаю firewalld:

```
systemctl stop iptables.service
```

```
systemctl start firewalld.service
```

Затем блокирую возможность запуска iptables:

```
systemctl mask iptables.service
```

Команда создаёт символическую ссылку:

```
/etc/systemd/system/iptables.service → /dev/null
```

Файл юнита в каталоге `/etc/systemd/system` имеет более высокий приоритет, чем файл из `/usr/lib/systemd/system`, поэтому обращение юнита к `/dev/null` полностью блокирует его запуск.

Проверяю действие маскирования попыткой запуска:

```
systemctl start iptables
```

В результате `systemd` возвращает ошибку:

```
Failed to start iptables.service: Unit iptables.service is masked.
```

Дополнительно пытаюсь добавить службу в автозапуск:

```
systemctl enable iptables
```

Операция также завершается ошибкой, поскольку замаскированный юнит не может быть включён:

```
Failed to enable unit: Unit /etc/systemd/system/iptables.service is masked
```

Следовательно, маскирование является более строгим механизмом блокировки, чем `disable`: отключённый юнит можно запустить вручную, тогда как замаскированный юнит нельзя ни запустить обычным способом, ни добавить в автозапуск до снятия маски (см. рис. Рисунок 2.11).

```
root@mohammedlabsi:/home/mohammedlabsi#  
root@mohammedlabsi:/home/mohammedlabsi# systemctl stop iptables.service  
root@mohammedlabsi:/home/mohammedlabsi# systemctl start firewalld.service  
root@mohammedlabsi:/home/mohammedlabsi# systemctl mask iptables.service  
Created symlink '/etc/systemd/system/iptables.service' → '/dev/null'.  
root@mohammedlabsi:/home/mohammedlabsi# systemctl start iptables  
Failed to start iptables.service: Unit iptables.service is masked.  
root@mohammedlabsi:/home/mohammedlabsi# systemctl enable iptables  
Failed to enable unit: Unit /etc/systemd/system/iptables.service is masked  
root@mohammedlabsi:/home/mohammedlabsi#
```

Рисунок 2.11: Маскирование службы `iptables` и проверка блокировки её запуска

2.3 Работа с изолируемыми целями systemd

Перехожу в каталог, содержащий системные unit-файлы:

```
cd /usr/lib/systemd/system
```

Для поиска целей, допускающих изоляцию, выполняю:

```
grep Isolate *.target
```

В результате отображаются target-юниты, в которых присутствует параметр:

```
AllowIsolate=yes
```

Среди найденных целей находятся `graphical.target`, `multi-user.target`, `rescue.target`, `poweroff.target`, `reboot.target`, `emergency.target` и ряд других.

Параметр `AllowIsolate=yes` означает, что к данной цели разрешено переключиться с помощью команды `systemctl isolate`. При изоляции `systemd` запускает зависимости выбранной цели и останавливает юниты, которые для неё не требуются (см. рис. Рисунок 2.12).

```
root@mohammedlabsi:/home/mohammedlabsi# cd /usr/lib/systemd/system
root@mohammedlabsi:/usr/lib/systemd/system# grep Isolate *.target
ctrl-alt-del.target:AllowIsolate=yes
default.target:AllowIsolate=yes
emergency.target:AllowIsolate=yes
exit.target:AllowIsolate=yes
graphical.target:AllowIsolate=yes
halt.target:AllowIsolate=yes
initrd-switch-root.target:AllowIsolate=yes
initrd.target:AllowIsolate=yes
kexec.target:AllowIsolate=yes
multi-user.target:AllowIsolate=yes
poweroff.target:AllowIsolate=yes
reboot.target:AllowIsolate=yes
rescue.target:AllowIsolate=yes
runlevel0.target:AllowIsolate=yes
runlevel1.target:AllowIsolate=yes
runlevel2.target:AllowIsolate=yes
runlevel3.target:AllowIsolate=yes
runlevel4.target:AllowIsolate=yes
runlevel5.target:AllowIsolate=yes
runlevel6.target:AllowIsolate=yes
soft-reboot.target:AllowIsolate=yes
system-update.target:AllowIsolate=yes
root@mohammedlabsi:/usr/lib/systemd/system#
```

Рисунок 2.12: Поиск целей systemd, поддерживающих изоляцию

Для перехода в режим восстановления выполняю:

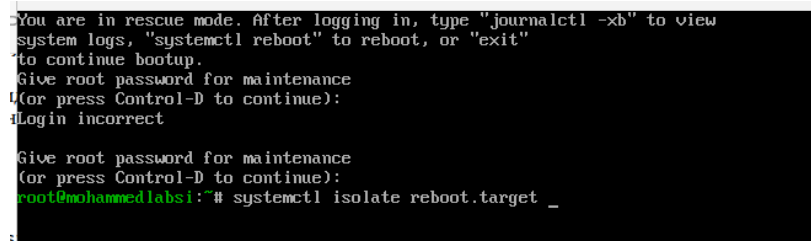
```
systemctl isolate rescue.target
```

После переключения система переходит в `rescue mode` и запрашивает пароль пользователя `root` для выполнения административных операций. После успешной аутентификации появляется консоль суперпользователя.

Для перезагрузки системы через изолируемую цель выполняю:

```
systemctl isolate reboot.target
```

Команда переключает систему на цель `reboot.target`, предназначенную для завершения текущего сеанса работы и перезапуска операционной системы (см. рис. Рисунок 2.13).



```
You are in rescue mode. After logging in, type "journalctl -xb" to view
system logs, "systemctl reboot" to reboot, or "exit"
to continue bootup.
Give root password for maintenance
(or press Control-D to continue):
Login incorrect
Give root password for maintenance
(or press Control-D to continue):
root@mohammedlabsi: # systemctl isolate reboot.target _
```

Рисунок 2.13: Работа в `rescue mode` и переход к цели `reboot.target`

2.4 Настройка цели `systemd` по умолчанию

После получения полномочий администратора определяю цель, используемую системой по умолчанию:

```
systemctl get-default
```

Первоначально результатом является:

```
graphical.target
```

Следовательно, система настроена на загрузку графического режима.

Для изменения режима загрузки устанавливаю в качестве цели по умолчанию `multi-user.target`:

```
systemctl set-default multi-user.target
```

При выполнении команды удаляется прежняя ссылка `/etc/systemd/system/default.target` и создаётся новая:

`/etc/systemd/system/default.target` → `/usr/lib/systemd/system/multi-user.target`

Проверяю результат:

`systemctl get-default`

Команда выводит:

`multi-user.target`

Следовательно, при следующей загрузке система должна перейти в многопользовательский текстовый режим без запуска графической среды (см. рис. Рисунок 2.14).

```
mohammedlabsi@mohammedlabsi:~$ sudo su
[sudo] password for mohammedlabsi:
root@mohammedlabsi:/home/mohammedlabsi# systemctl get-default
graphical.target
root@mohammedlabsi:/home/mohammedlabsi# systemctl set-default multi-user.target
Removed '/etc/systemd/system/default.target'.
Created symlink '/etc/systemd/system/default.target' → '/usr/lib/systemd/system/multi-user.target'.
root@mohammedlabsi:/home/mohammedlabsi# systemctl get-default
multi-user.target
root@mohammedlabsi:/home/mohammedlabsi#
```

Рисунок 2.14: Изменение цели по умолчанию на `multi-user.target`

После перезагрузки операционная система действительно загружается в текстовом режиме. На экране отображается консоль входа Rocky Linux, после чего выполняю вход под учётной записью `root`.

Проверяю текущую цель по умолчанию:

`systemctl get-default`

Результат:

`multi-user.target`

Это подтверждает применение выполненной ранее настройки.

Для возврата к загрузке графического режима выполняю:

`systemctl set-default graphical.target`

При этом ссылка `/etc/systemd/system/default.target` перенаправляется на:

`/usr/lib/systemd/system/graphical.target`

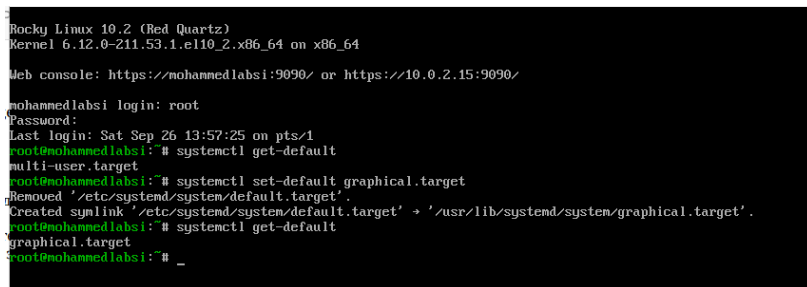
Повторно выполняю:

`systemctl get-default`

В результате отображается:

`graphical.target`

Следовательно, в качестве цели загрузки по умолчанию снова установлен графический режим (см. рис. Рисунок 2.15).



```
Rocky Linux 10.2 (Red Quartz)
Kernel 6.12.0-211.53.1.el10.2.x86_64 on x86_64

Web console: https://mohammedlabsi:9090/ or https://10.0.2.15:9090/

mohammedlabsi login: root
Password:
Last login: Sat Sep 26 13:57:25 on pts/1
root@mohammedlabsi:~# systemctl get-default
multi-user.target
root@mohammedlabsi:~# systemctl set-default graphical.target
Removed '/etc/systemd/system/default.target'.
Created symlink '/etc/systemd/system/default.target' → '/usr/lib/systemd/system/graphical.target'.
root@mohammedlabsi:~# systemctl get-default
graphical.target
root@mohammedlabsi:~# _
```

Рисунок 2.15: Проверка текстового режима и восстановление `graphical.target` как цели по умолчанию

2.5 Контрольные вопросы

1. Что такое юнит (unit)? Приведите примеры.

Юнит (unit) — это объект, которым управляет система инициализации **systemd**. Юнит описывается специальным unit-файлом и может представлять службу, цель, устройство, точку монтирования, сокет и другие системные объекты.

Тип юнита определяется расширением его файла. Например, **vsftpd.service** и **firewalld.service** являются сервисными юнитами, **multi-user.target** и **graphical.target** — целевыми юнитами, **home.mount** — юнитом точки монтирования, а файлы с расширением **.socket** представляют сокеты.

Управление юнитами осуществляется преимущественно с помощью команды **systemctl**.

2. **Какая команда позволяет вам убедиться, что цель больше не входит в список автоматического запуска при загрузке системы?**

Для проверки состояния автоматического запуска юнита можно использовать команду **systemctl is-enabled**.

Например: **systemctl is-enabled имя.target**.

Если цель исключена из автоматического запуска, команда выведет состояние **disabled**. Для цели загрузки системы по умолчанию дополнительно используется команда **systemctl get-default**, которая показывает, какая цель в настоящий момент установлена как цель по умолчанию.

3. **Какую команду вы должны использовать для отображения всех сервисных юнитов, которые в настоящее время загружены?**

Для отображения сервисных юнитов используется команда **systemctl list-units --type=service**.

Она выводит сервисные юниты, загруженные системой systemd, и показывает их состояние.

Если необходимо включить в вывод также неактивные загруженные сервисные юниты, можно выполнить: **systemctl list-units --type=service --all**.

4. **Как создать потребность (wants) в сервисе?**

Зависимость типа **Wants** можно создать командой **systemctl add-wants**.

Например: **systemctl add-wants multi-user.target vsftpd.service**.

В результате для указанной цели создаётся зависимость от сервиса. Фактически systemd создаёт соответствующую символическую ссылку в каталоге **multi-user.target.wants**.

При обычном добавлении сервиса в автозапуск также часто используется команда **systemctl enable имя.service**, которая создаёт ссылки в соответствии с параметром **WantedBy** секции **[Install]** unit-файла.

5. **Как переключить текущее состояние на цель восстановления (rescue target)?**

Для перехода системы в режим восстановления необходимо выполнить команду **systemctl isolate rescue.target**.

Systemd активирует **rescue.target** и её зависимости, одновременно останавливая юниты, которые для режима восстановления не требуются. После перехода в этот режим система может запросить пароль пользователя **root** для выполнения административных операций.

6. **Поясните причину получения сообщения о том, что цель не может быть изолирована.**

Не каждая цель systemd разрешает выполнение операции **isolate**. Чтобы цель могла быть изолирована, в её unit-файле должен присутствовать параметр **AllowIsolate=yes**.

Если этого параметра нет либо для него установлено значение **no**, команда **systemctl isolate имя.target** завершится ошибкой, сообщающей, что операция изоляции для данной цели запрещена.

Найти цели, поддерживающие изоляцию, можно, например, командой **grep Isolate /usr/lib/systemd/system/*.target**.

7. **Вы хотите отключить службу systemd, но, прежде чем сделать это, вы хотите узнать, какие другие юниты зависят от этой службы. Какую команду вы бы использовали?**

Для просмотра юнитов, которые зависят от выбранной службы, используется команда **systemctl list-dependencies имя.service --reverse**.

Например: **systemctl list-dependencies vsftpd.service --reverse**.

Параметр **--reverse** изменяет направление просмотра зависимостей и показывает не те юниты, от которых зависит указанный сервис, а те юниты, которые сами зависят от него. Это позволяет оценить возможные последствия отключения службы.

3 Заключение

В ходе лабораторной работы изучены основные возможности системы инициализации `systemd` и способы управления системными службами в Rocky Linux. Рассмотрены операции запуска, остановки, проверки состояния и настройки автоматического запуска сервисов с помощью команды `systemctl`.

На примере службы `vsftpd` освоена установка сервиса, его запуск, добавление в автозагрузку и удаление из неё. Рассмотрено создание символических ссылок в каталоге `/etc/systemd/system/multi-user.target.wants/`, а также изучены прямые и обратные зависимости юнитов с помощью команд `systemctl list-dependencies` и `systemctl list-dependencies --reverse`.

На примере служб `firewalld` и `iptables` изучен механизм конфликтов между юнитами `systemd`. Рассмотрена директива `Conflicts` в `unit`-файле `firewalld.service` и проверено поведение системы при попытке одновременного запуска конфликтующих служб. Дополнительно освоено маскирование юнита с помощью `systemctl mask`, при котором создаётся символическая ссылка на `/dev/null`, полностью запрещающая запуск и добавление службы в автозагрузку.

Также изучены цели `systemd` и механизм их изоляции. С помощью параметра `AllowIsolate=yes` определены цели, которые допускают переключение командой `systemctl isolate`. Выполнен переход системы в режим восстановления `rescue.target` и рассмотрена перезагрузка через цель `reboot.target`.

В завершение изучена настройка цели загрузки системы по умолчанию. С помощью команд `systemctl get-default` и `systemctl set-default` выполнено пе-

реключение между `multi-user.target` и `graphical.target`. Проверена загрузка системы как в текстовом, так и в графическом режиме.